

Stack

Page No.

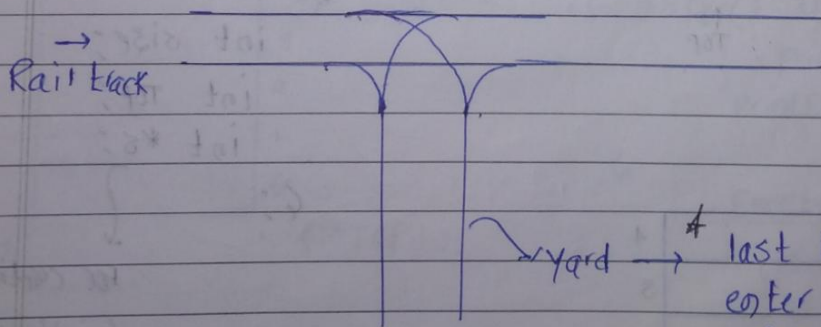
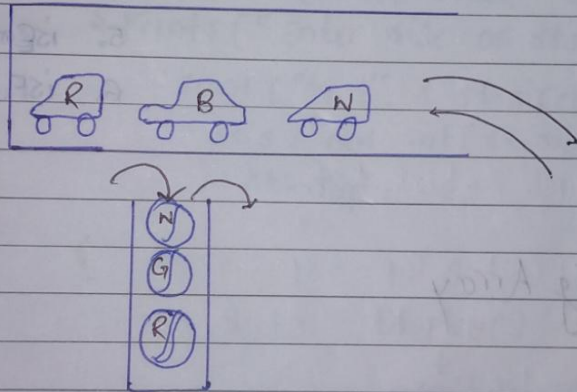
Date: / /

1. Introduction / Application
2. ADT stack

1. Introduction / Application

stack $\rightarrow$ stack is collection of element which follow discipline LIFO.

LIFO $\rightarrow$ Last-in First-out



Recursion
some iterative } $\rightarrow$ Uses stack

2. ADT

	4
25	3 ← TOP=3
20	2
16	1
10	0

ADT stack

Data:

1. Space For storing elements
2. Top pointer

Operations:

1. Push(x)
2. Pop()
3. Peek(index)
4. StackTop()
5. isEmpty()
6. isFull()

Stack use 1. Array For storing elements
2. Linked List

Stack Using Array

size=5

5	10	5	7		
-1	0	1	2		

↑
TOP

Struct stack

{

int size;

int TOP;

int *s;

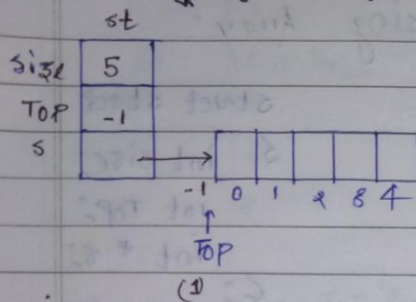
};

For creating
variable size
array

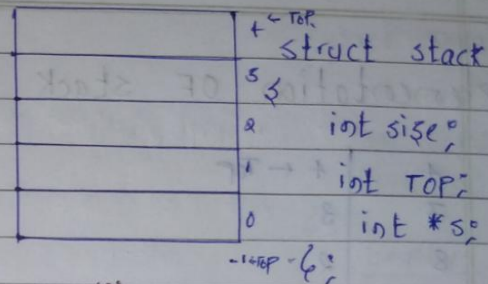
	4
	3
	2
	1
	0

-1 ← TOP

→ Actually stack created in following way using array



But we use 2nd one for visualisation process thinking size=5

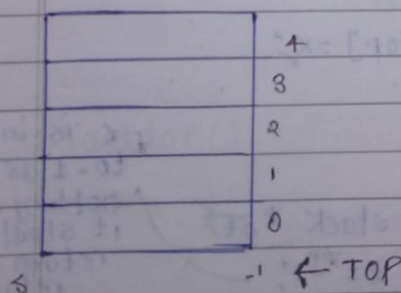


```
int main
{
    struct stack st;
    void create(struct stack *st)
    {
        printf("Enter size of stack");
        scanf("%d", &st.size);
        st.s = new int[st.size];
        st.TOP = -1;
    }
}
```

Display()

```
void Display(struct stack st)
{
    int i;
    for(i = st.TOP; i >= 0; i--)
        printf("%d", st.s[i]);
    printf("\n");
}
```

size=5



Empty

IF (TOP == -1)

Full

IF (TOP == size - 1)

Implementation of stack using Array

size=5

4	4 ← Top
7	3
8	2
15	1
10	0
	-1

struct stack

int size;

int Top;

int * s;

{

Empty

IF (TOP == -1)

Full

IF (TOP == size-1)

Push()

void push (stack *st, int x)

{

IF (st->TOP == st->size-1)

printf("stack overflow");

→ O(1)

else

st->TOP++;

st->s[st->TOP] = x;

{

{

POP()

size=5

	4
	3
	2
	1
10	0
	-1

O(1) ←

int pop (stack *st)

{

IF (st->TOP == -1)

printf("stack underflow");

else

x = st->s[st->TOP];

st->TOP--;

{

return x;

{

x is initialised to -1, as IF nothing deleted it should return -1 as return is int type

if nothing deleted then return -1

size = 5

peek()

	4
20	3 ← Top = 3
8	2
15	1
10	0

↓

Pos Index = Top - Pos + 1

1	→ 3 = 3 - 1 + 1
2	2 = 3 - 2 + 1
3	1 = 3 - 3 + 1
4	0 = 3 - 4 + 1

int peek (stack st, int pos)
int x = -1;

IF (Top - Pos + 1 < 0)

PrintF ("Invalid position")

else

x = st.s [st.Top - Pos + 1];

return x;

IF returning -1 means
our pos is wrong
or value not found

StackTop()

int stackTop(stack st)

{

IF (st.top == -1)

return -1;

else

return st.s [st.Top];

is empty()

```
int isEmpty(stack st)
```

```
{
```

```
    if (st.Top == -1)
```

```
        return 1;
```

```
    else
```

```
        return 0;
```

```
}
```

is Full()

```
int isFull (stack st)
```

```
{
```

```
    if (st.Top == st.size - 1)
```

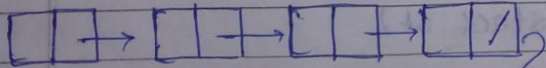
```
        return 1;
```

```
    else
```

```
        return 0;
```

```
}
```

Stack using LinkedList

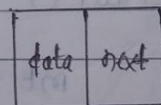


* inserting & deleting from this side is of $O(1)$

inserting & deleting from this side is of $O(1)$

↓
so we should insert & delete from left side

Node

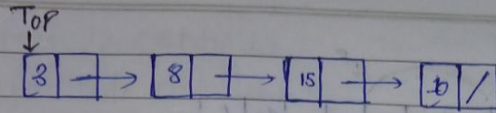


struct Node

int data;

struct Node *next;

};



Empty $\rightarrow$ TOP $\rightarrow$ x
 IF (TOP == NULL)

* If heap is full & no new node being created so it will be null

Full
 Node *t = new Node
 IF (t == NULL)

Push()



void push(int x)

{

Node *t = new Node;

IF (t == NULL)

printf("stack overflow");

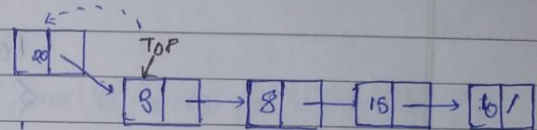
else

t->data = x;

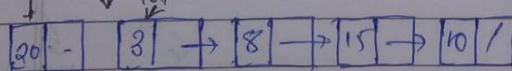
t->next = TOP;

TOP = t;

}



POP()



int POP()

{ Node *p;

int x = -1;

IF (TOP == NULL)

printf("stack is Empty
 stack underflow");

else

p = TOP;

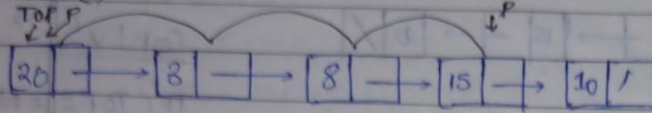
TOP = TOP->next;

x = p->data;

return x;

in this case it will return -1

Page No. _____
Date: / /



For Pos = 4 → only 3 movement

int Peek (int Pos)

```

{
    int i;
    Node *P = Top;

```

For (i=0; P != NULL & i < Pos-1; i++)

```

{
    P = P->next;

```

```

{
    IF (P != NULL)

```

```

        return P->data;
    else

```

```

        return -1;
    }
}

```

StackTop()

int StackTop()

```

{
    IF (TOP != NULL)
        return TOP->data;
    return -1;
}

```

isEmpty()

int isEmpty()

```

{
    return TOP ? 0 : 1;
}
// means TOP != NULL

```

isFull()

int isFull()

```

{
    Node *t = new Node;
    int r = t->data;
    free(t);
    return r;
}
// means top = NULL

```

→ #include <stdio.h>
#include <stdlib.h>

struct Node

{
int data;
struct Node *next;
}*top = NULL

You can also create top inside main
* Pass it as parameter

void push(int x)

{
struct Node *t;
t = (struct Node *) malloc(sizeof(struct Node));
if (t == NULL)
printf("stack is Full\n");
else
{
t->data = x;
t->next = top;
top = t;
}}

int pop()

{
struct Node *t;
if (top == NULL)
printf("stack is Empty\n");
else
{
t = top;
top = top->next;
x = t->data;
free(t);
}
return x;
}

Void Display()
 { struct Node *P;
 P = top

while(P != NULL)

{ printf("%d", P->data);

P = P->next;
 printf("\n");
 }

{

int main()

{ Push(10);

Push(20);

Push(30);

Display()

printf("%d", POP());
 }

{

Let's Code using C++

→ #include <iostream>
 using namespace std;

class Node

{ public:
 int data;

Node *next;

};

class stack
 { Private:
 Node *top;
 Public:
 stack() { top = NULL; }
 void push(int x) {
 int pop();
 void Display();
 }
 };

void stack::Push(int x)
 {
 Node *t = new Node;
 IF (t == NULL)
 cout << "stack is Full";
 else
 {
 t->data = x;
 t->next = top;
 top = t;
 }
 }

int stack::Pop()
 {
 IF (top == NULL)
 cout << "stack is empty";
 else
 {
 x = top->data;
 Node *t = top;
 top = top->next;
 delete t;
 }
 return x;
 }

```

void stack::Display()
{
    Node *P = top;
    while (P != NULL)
    {
        cout << P->data << " ";
        P = P->next;
    }
    cout << endl;
}

```

```

int main()
{
    stack stk;
    stk.Push(10);      → 30 20 10
    stk.Push(20);      → 30
    stk.Push(30);
    stk.Display();    cout << stk.Pop();
    return 0;
}

```

Application of Stack

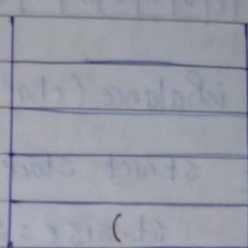
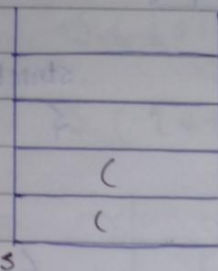
Paranthesis Matching → checking Parenthesis is balanced or not

$((a+b) * (c-d))$

↓
For every opening bracket there should be closing bracket

★ $(a+)b(*c-d) \rightarrow$ Improper Parenthesis but balanced

Page No.

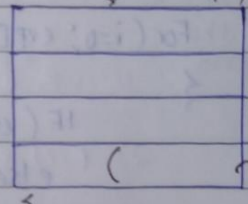
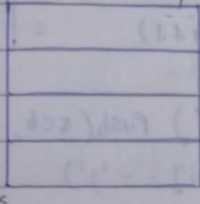
$$((a+b) * (c-d))$$
$$((a+b) * (c-d))$$


* For every opening bracket push-in & For every closing bracket push out & ignore other thing

$$((a+b) * (c-d))$$
$$((a+b) * (c-d))$$

↓ empty

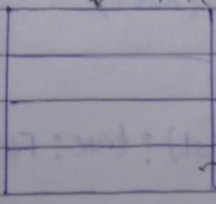
↓ not empty



→ ~~Keep~~ Care of this case

~~***~~ $((a+b) * (c-d))$

↓ empty (but above Parenthesis is not balanced)



→ Keep Care of this case

corner Case

M

struct stack

struct stack st;

```

2  int size;
   int Top;
   char *s;

```

```
st.size = strlen(exp);
```

$$SL \cdot TOP = -1;$$

```
st.s = new char[st.size];
```

6. 10 theory

int in

For ($i=0$; $\text{exp}[i] = (10^6; i++)$)

Process

size	13				
top	-1				
s		→			---

IF (exp[i] == '(') push(stack, exp[i]);

```
else if (exp[i] == '(')
```

specifically for
this case this
statement has been
written

```

if (isEmpty(st)) return false;
Pop(&st);

```

```
return isEmpty(st) ? true : false;
```

↓ ★
try to code by self for LinkedList. If
not able to do then see Resources (video)

More On parenthesis Matching

$\{ ([a+b] * [c-d]) / e \}$ → balanced

$\{ ([a+b] * [c-d]) / e \}$ → Not balanced

HINT

HINT For checking

ASCII ↓

40 (

diff=1

41)

→ below 50

Address

91 [

93]

diff=2

125 {

diff=2

125 {

OR

you can write so

many cond

[
(
{

IF (exp[i] == '{' || exp[i] == '(' || ...)

push(&st, exp[i])

+

IF (exp[i] == '{' || exp[i] == '(' || ...)

x = pop(&st)

* check x is matching
with type of bracket
or not

student exercise

↓

Infix to postfix Conversion

1. What is postfix
2. Why postfix
3. Precedence
4. Manual conversion

1. What is postfix

1. Infix: operand operator operand

eg:- $a+b$

2. Prefix: OP. opnd. opnd

eg:- $+ab$

3. Postfix:- opnd. opnd. OP.

eg:- abt

2. Why postfix & prefix

$$\frac{8+3}{6} * \frac{(9-6)}{5} / 2^2 + \frac{6}{2}$$

6 5 + 3 2 7 +

⤴ If we write program in infix then we have to scan above expression multiple times. It is more time consuming.

PostFix $\rightarrow 8 \ 3 \ 96 \ -2^2 / * + 62 / +$

↓
We can perform above expression in
Just single scan.

3. InFix to postFix Conversion

- ① $a+b*c$ ② $a+b+c*d$

★ For Conversion

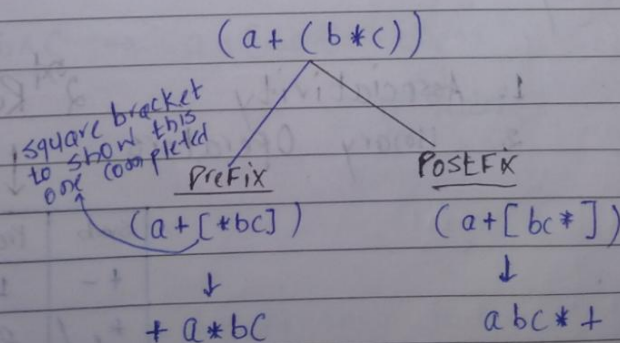
rule:- ① Whenever you write an expression actually you should make it Fully Parenthesise. (When expression is not

② Parenthesised compiler will automatically Convert it not fully but logically) by using Precedence

Symbol	Precedence
$+, -$	1
$*, /$	2
$()$	3

*** ② Precedence doesn't mean order of execution. What operation will be performed it's a different thing. Precedence & associativity are meant for Parenthesitation. Not for which will execute first

① $a+b*c \rightarrow a+(b*c) \rightarrow (a+(b*c))$



② $a + b + c * d$

PostFix

 $a + b + c * d$ $a + b + [c * d]$ $[a + b] + [c * d]$ Pre: $++ab * cd$

PostFix

 $a + b + c * d$ $a + b + [c * d]$ $[a + b] + [c * d]$ Post: $ab + cd * +$

If precedences are same take them from left to right

here +, +

③ $(a + b) * (c - d)$

PostFix

 $(a + b) * (c - d)$ $[a + b] * (c - d)$ $[a + b] * [c - d]$ Pre: $* + ab - cd$

PostFix

 $(a + b) * (c - d)$ $[a + b] * (c - d)$ $[a + b] * [c - d]$ Post: $ab + cd - *$

Associativity

Only 2 rule in postfix

1. Associativity

2. Unary Operators

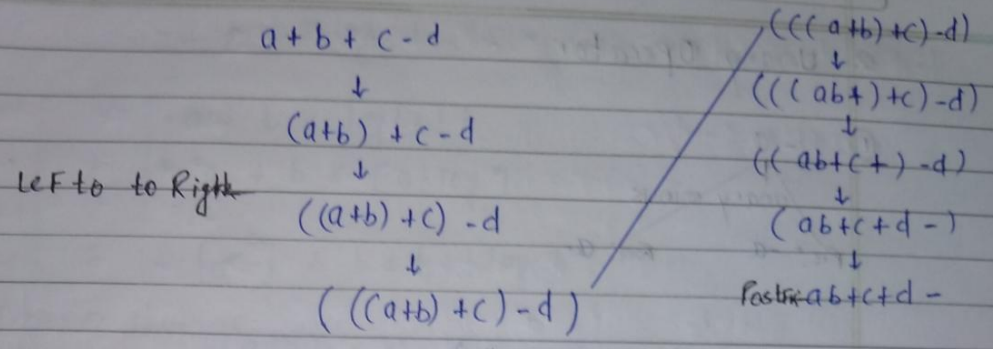
2nd Rule of Postfix

1. Associativity

$\rightarrow$ some author use $\&$ symbol also

Symbol	Pre	Assd
+ -	1	L-R
*, /	2	L-R
^	3	R-L
~	4	R-L
()	5	L-R

Power $\leftarrow$ Unary $\leftarrow$



R to L

$$a = b = c = 5 \rightarrow a = b = (c = 5) \rightarrow a = (b = (c = 5)) \rightarrow (a = (b = (c = 5)))$$

$$(a = (b = (c = 5))) \rightarrow (a = (b = (c = 5))) \rightarrow (a = (b = (c = 5))) \rightarrow (a = (b = (c = 5)))$$

Postfix

1st Rule of Postfix

* Postfix depends on Parenthesisation & Parenthesisation depends on precedence & associativity.

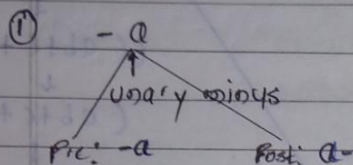
$$a \wedge b \wedge c \rightarrow a \wedge (b \wedge c) \rightarrow (a \wedge (b \wedge c)) = (a)^{(b^c)}$$

According to R to L associativity
(a^b)^c IF we follow L to R associativity, result are completely different

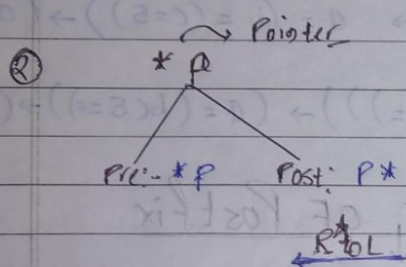
$$a \wedge b \wedge c \rightarrow a \wedge [b \wedge c] \rightarrow abc \wedge \wedge$$

Postfix

2. Unary Operator

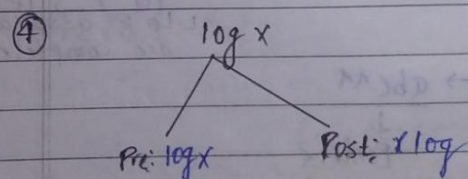
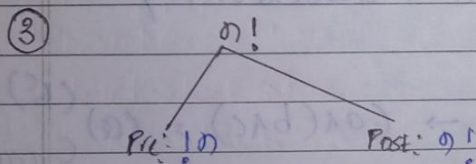


$$--a \rightarrow -(-a) \rightarrow (-(-a))$$



$$**P \rightarrow *(*P) \rightarrow (*(*P))$$

Unary Operators in languages generally have highest precedence & R to L associativity



Ex: $-a + b * \log n!$

$-, \log, ! \rightarrow$ have same precedence

$$-a + b * \log n!$$

$$-a + b * \log[n!]$$

$$-a + b * [n! \log]$$

$$[a-] + b * [n! \log]$$

$$[a-] + [b n! \log *]$$

$$a - b n! \log * +$$

InFix to PostFix using stack Method 1

$$a + b * c - d / e$$

tracing logic :-

Symbol	Pre	Assoc
+	1	L-R
*	2	L-R

*
Visualize it
like from top
to bottom

Symbol	Stack	PostFix
a	-	a
+	+	a
b	+	ab
*	*, +	ab
c	*, +	abc
-	-	abc*+
d	-	abc*+d
/	/, -	abc*+d
e	/, -	abc*+de
End of expression		abc*+de/-

Stack

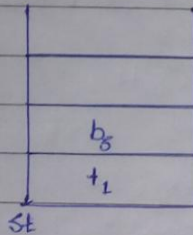
PostFix
↓
ab

*
If already higher
or equal symbol present
in stack then Pop-out
otherwise Pop-in &
at the end Pop-out
all.

InFix to PostFix using stack Method 2

$a + b * c - d / e$

↑



PostFix: $a * b +$

Syb	Pre	Asso
$+, -$	1	L-R
$*, /$	2	L-R
$^$	3	R-L

→ * everything higher or equal present in stack pop-out otherwise pop-in & at the end pop-out all.

Program OF InFix to PostFix Conversion

InFix:

a	+	b	*	c	-	d	/	e	
0	1	2	3	4	5	6	7	8	9

→ for returning PostFix
char * Convert(char * InFix)

{ struct stack st;

// initialised

char * PostFix = new char[strlen(InFix) + 1];

int i = 0, j = 0;

while (InFix[i] != '\0')

{

if (isOperand(InFix[i]))

PostFix[j++] = InFix[i];

else

if (Pre(InFix[i]) > Pre(stackTop(st)))

Push(st, InFix[i]);

else PostFix[j++] = Pop(st);

}

}

for out character

int isOperand(char x)

if (x == '+' || x == '-' || x == '*' || x == '/')

return 0;

else

return 1;

int Pre(char x)

if (x == '+' || x == '-')

return 1;

else if (x == '*' || x == '/')

return 2;

return 0;

→ if empty/empty

it will return 0

return 0 if any other

symbol is present

While (! isEmpty(st))

{

PostFix[j++] = Pop(&st);

}

PostFix[j] = '\0';

return postFix;

}

→ #include <stdio.h>

#include <stdlib.h>

struct Node

{

char data;

struct Node *next;

*top = NULL;

void push(char x)

{

!

}

char Pop()

{ struct Node *t; char x = -1; IF(top == NULL) PF("empty")

else {

return x;

}

void display()

{

!

}

int isBalanced(char *exp)

{

int i;

for(i=0; exp[i] != '\0'; i++)

{

if(exp[i] == '(')

push(exp[i]);

else if(exp[i] == ')')

{

if(top == NULL)

return 0;

pop();

}

}

if(top == NULL)

return 1;

else

return 0;

}

int pre(char x)

{

if(x == '+' || x == '-')

return 1;

else if(x == '*' || x == '/')

return 2;

else

return 0;

}

int isOperand(char x)

{

if(x == '+' || x == '-' || x == '*' || x == '/')

return 0;

else

return 1;

}

char *InToPost (char *inFix)

int i=0, j=0;

char *PostFix;

+1 for #

int len = strlen (inFix);

PostFix = (char *) malloc ((len + 2) * sizeof(char));

while (inFix[i] != '\0')

if (isoperand (inFix[i]))

PostFix[j++] = inFix[i++];

else

if (Pre(inFix[i]) > Pre(top->data))

Push (inFix[i++]);

else

PostFix[j++] = Pop();

while (top != NULL)

PostFix[j++] = Pop();

PostFix[j] = '\0';

return PostFix;

int main()

char *inFix = "a+b*c";

Push('#'); → If stack will empty then error will come
so to manage it

char *PostFix = InToPost (inFix);

printf ("%s", PostFix); → abc*+##

return 0;

In Fix to postFix With Associativity and parenthesis in order

$$(a+b) * c - d \wedge e \wedge f$$

→ right to Left associativity

$$(a+b) * c) - d \wedge e \wedge f$$

$$([ab+] * c) - d \wedge e \wedge f$$

$$[ab+c*] - d \wedge e \wedge f$$

$$[ab+c*] - d \wedge [ef\wedge]$$

$$[ab+c*] - [def\wedge\wedge]$$

$$[ab+c*def\wedge\wedge-]$$

Symbol	Outstack pre	instack pre
+,-	1	2
*,/	3	4
^	6	5
(	7	0
)	0	?

→ From L to R increase the Precedences & For R to L decrease the Precedences in this way you can add more operator in table. this is your table.

Procedure:-

$$((a+b) * c) - d \wedge e \wedge f$$

+
(
(

PostFix
ab

$$((a+b) * c) - d \wedge e \wedge f$$

+
(
(
)
)

PostFix.
ab+

* (bracket pop-out but don't add in PostFix

→ In continuing above process we will get $abc+defm$ as PostFix.

↓
student Exercise

Evaluation OF PostFix expression

$$3 * 5 + 6 / 2 - 4$$

$$15 + 3 - 4$$

$$= 18 - 4 = 14$$

$$3 * 5 + 6 / 2 - 4$$

$$\text{PostFix} \rightarrow 35 * 62 / + 4 -$$

↑ ↓ Evaluation will be done in only one scan.

5	
3	

st
if you get operator then pop-out two value operate them or Evaluate them & then Pop-in

15	

st

$$3 * 5 = 15$$

$$6 / 2 = 3$$

$$15 + 3 = 18$$

$$18 - 4 = 14$$

will be Popped in

Tracing

symbol	stack	Operation
3	3	
5	5, 3	
*	15	$3 * 5 = 15$
6	6, 15	
2	2, 6, 15	
/	3, 15	$6 / 2 = 3$
+	18	$15 + 3 = 18$
4	4, 18	
-	14	$18 - 4 = 14$

$$x = 6 + 5 + 3 * 4$$

↑
* Or believe it will operate First

$$x = 6 + 5 + 3 * 4 =$$

11
x

$$6 + 5 = 11$$

↑
* But here + operated Firstly
So precedence doesn't mean
For which will operate
First. Precedence decide which
will Parenthesise First

$$x = ((6 + 5) + (3 * 4))$$

↑
* After Parenthesising, Now we can
say it will operate First.

Program For Evaluation of postfix

Postfix	5	5	*	6	2	/	+	4	-	10
	0	1	2	3	4	5	6	7	8	9

```
int Eval (char *PostFix)
```

```
struct stack st;
```

// initialise (make top.1 ∞ -)

$$\text{int } i, x1, x2, r;$$

For ($i = 0$; PostFix[i] = '0' ; $i++$)

```
if (isOperand(PostFix[i]))
```

Push (ast, PostFix[i] - '0')

else

First element out

made it as second
opening

Switch (PostFix[i])

Case 't': $r = x_1 + x_2$; $\text{Push}(\text{get}_t, r)$; break;

$\text{QSE}(-1) \neq X(1-X_2)$ " TM (1)

Case '*': $r = x_1 * x_2$ //

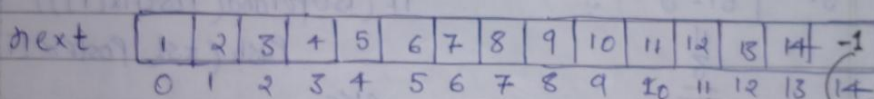
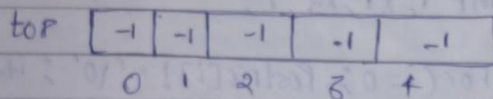
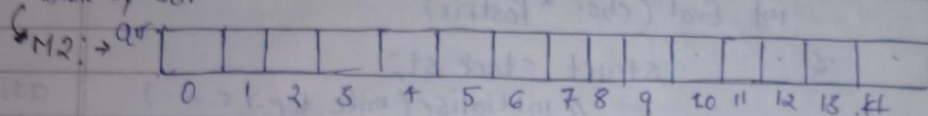
Case 1: $x_1 \neq x_2$ " "

↓ Infix to postfix Conversion is done for designing a compiler. If you are making compiler then you can use it.

1. Implement K stack in a single array efficiently

$M-1$ divide array into K slots $\rightarrow$ there is wastage of space also

think by self



array is full

loop

Free = [0]

Nearest

2. Next greater element $\rightarrow$ with this pattern there is 7 medium questions

$\rightarrow$ Ex: - 6 2 5 4 6

Ans $\rightarrow$ -1 5 6 6 -1

HINT: - use stack

3. implement Minimum element in stack without extra space
In $O(1)$ time Complexity

iently

f

school

is full

er

is

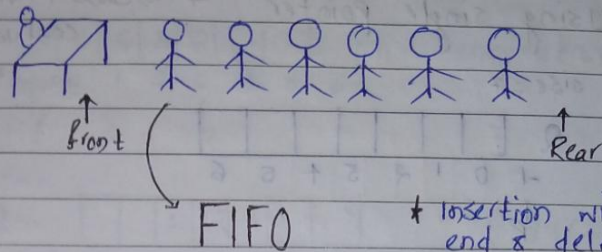
are

(1) (2) (3) (4) (5) (6) (7) (8) (9) (10) (11) (12) (13) (14) (15) (16) (17) (18) (19) (20) (21) (22) (23) (24) (25) (26) (27) (28) (29) (30) (31) (32) (33) (34) (35) (36) (37) (38) (39) (40) (41) (42) (43) (44) (45) (46) (47) (48) (49) (50) (51) (52) (53) (54) (55) (56) (57) (58) (59) (60) (61) (62) (63) (64) (65) (66) (67) (68) (69) (70) (71) (72) (73) (74) (75) (76) (77) (78) (79) (80) (81) (82) (83) (84) (85) (86) (87) (88) (89) (90) (91) (92) (93) (94) (95) (96) (97) (98) (99) (100)

STL

1. `empty()`
2. `size()`
3. `front()`, `back()`
4. `push()`, `pop()`
- 5.

Queues



* Insertion will be from rear end & deletion will be done from Front end.

Queue ADT

Data:

1. Space For storing elements
2. Front - For deletion
3. Rear - For insertion

Operations:

1. enqueue(x)
2. dequeue()
3. isEmpty()
4. isFull()
5. First()
6. last()

* Queue can be implemented using physical data structures

1. Array
2. Linked list

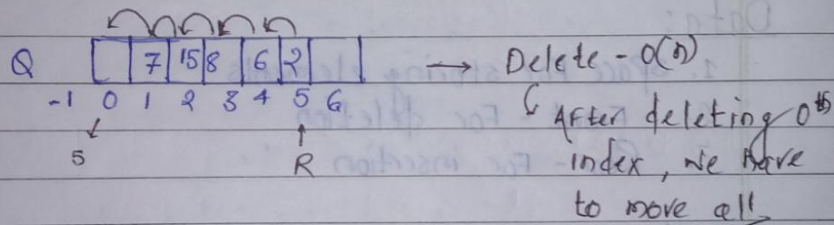
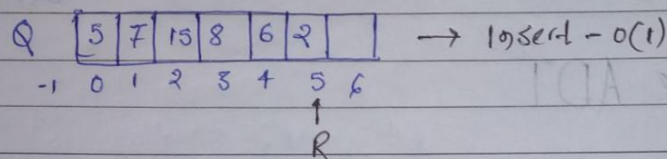
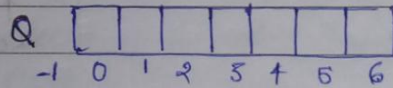
Queue Using Array

1. Queue Using - single pointer
2. Queue Using - Front and rear
3. Drawbacks of Queue Using Array

1. Queue Using single pointer

means index pointer doesn't confuse by mixing with address

size = 7

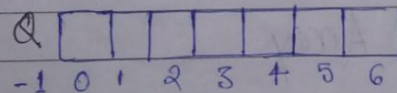


2. Queue using Front and rear



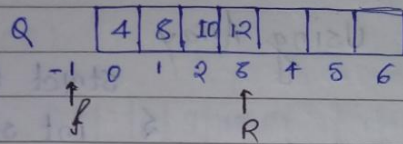
After deleting one man we can move the counter instead of moving all the man

size = 7

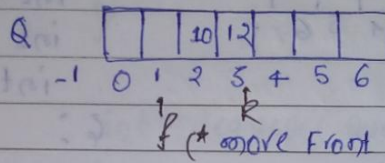


Initially

front = rear = -1



enqueue - $O(1)$



dequeue - $O(1)$

(* move Front & delete one element)

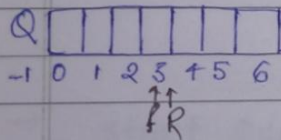
* Notice that Rear is on element & Front is before one element [imagine it just like counter]

initially

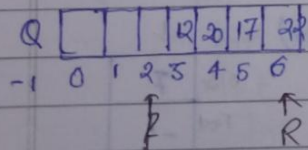
Front = Rear = -1

Empty

IF (Front == Rear)



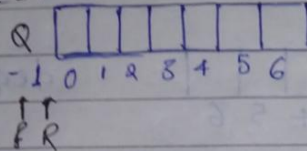
Full



IF (Rear == size - 1)

Implementing Queue Using Array

size = 7



struct Queue

```
{
    int size;
    int Front;
    int Rear;
    int *Q;
};
```

int main()

```
{
    struct Queue q;
```

size	7
Front	-1
Rear	-1
Q	→ [] [] [] [] [] [] []

```
    printf("Enter size");
```

```
    scanf("%d", &q.size);
```

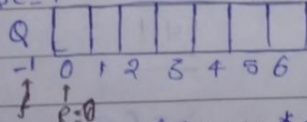
```
    q.Q = (int *) malloc(q.size * sizeof(int));
```

```
    q.Front = q.Rear = -1;
```

```
}
```

enqueue()

size = 7



void enqueue (Queue *q, int x)

```
{
    if (q->Rear == q->size - 1)
```

```
        printf("Queue is full");
```

```
    else
```

```
        q->Rear++;
```

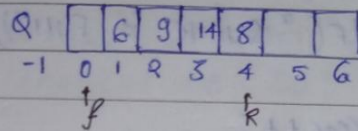
```
        q->Q[q->Rear] = x;
```

```
}
```

```
}
```

size	7
Front	-1
Rear	0
Q	→ [x] [] [] [] [] [] []

Deque() {



int dequeue(Queue *q)

{ int x = -1;

if (q->Front == q->Rear)

printf("Queue is Empty");

else

{ q->Front++;

x = q->Q[q->Front];

return x;

}

→ struct Queue

{ int size;

int Front;

int rear;

int *Q;

};

void create(struct Queue *q, int size)

{ q->size = size;

q->Front = q->rear = -1;

q->Q = (int *) malloc(q->size * sizeof(int));

}

void enqueue(int x)

{

if (q->rear == q->size - 1)

printf("Queue is Full");

else

{

q->rear++;

q[q->rear] = x;

}

}

int deque(struct Queue *q)

{

int x = -1;

if (q->front == q->rear)

printf("Queue is empty\n");

else

{

q->front++;

x = q->q[q->front];

}

return x;

}

void Display(struct Queue q)

{

int i;

for (i = q.front + 1; i <= q.rear; i++)

printf("%d", q.q[i]);

printf("\n");

}

int main()

{

struct Queue q;

Page No. _____
Date: / /

```
Create (89, 5);
```

```
enqueue (89, 10);
```

```
enqueue (89, 20);
```

```
enqueue (89, 30);
```

```
Display();
```

→ 10, 20, 30

```
printf("%d", dequeue(89));
```

→ 10

```
return 0;
```

```
}
```

C++
↓

```
#include <iostream>
```

```
using namespace std;
```

```
class Queue
```

```
{ private:
```

```
int front;
```

```
int rear;
```

```
int size;
```

```
int *Q;
```

```
public:
```

```
Queue() { front = rear = -1; size = 10; Q = new int[size]; }
```

```
Queue(int size) { front = rear = -1; this->size = size; Q = new int[size]; }
```

```
void enqueue(int x);
```

```
void dequeue();
```

```
void Display();
```

```
}
```

```
void Queue::enqueue(int x) {
    if (rear == size - 1) {
        printf("Queue Full\n");
    } else {
        rear++;
        Q[rear] = x;
        printf("%d\n", Q[rear]);
    }
}
```

```
int Queue::dequeue() {
    int x = -1;
    if (Front == rear) {
        printf("Queue is empty\n");
    } else {
        x = Q[Front + 1];
        Front++;
    }
    return x;
}
```

```
void Queue::Display() {
    for (int i = Front + 1; i <= rear; i++) {
        printf("%d\n", Q[i]);
    }
}
```

```
int main() {
    Queue q(5);
    q.enqueue(10);
    q.enqueue(10);
    q.enqueue(20);
    q.Display();
    return 0;
}
```

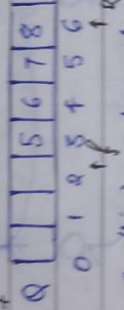
1. Drawbacks

2. Solution

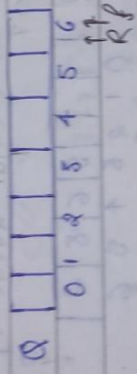
1. Resetting pointers
2. circular Queue

1. Drawbacks

size=7



- (i) We can't insert at spaces of deleted element
- (ii) One space can be used only once

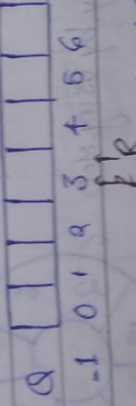


There may be situation when Q is empty as well as Full also

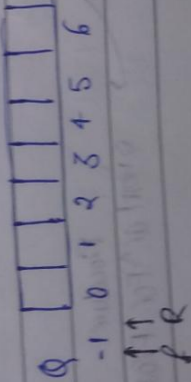
2. solution

1. Resetting pointer

size=7



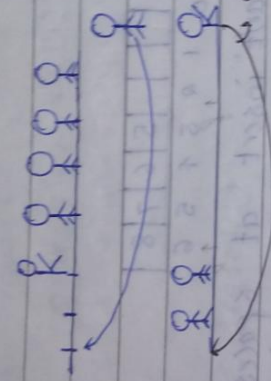
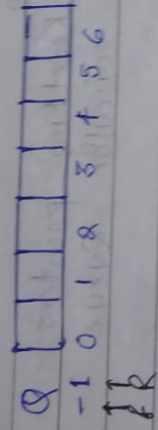
* Whenever front & rear become equal bring them to -1



* this method also doesn't guarantee of reusing of deleted spaces

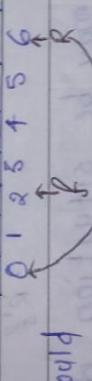
2. Circular Queue

size = 7



moving to initial after finishing

in circular queue



size = 7

When Front remains present that should remain empty

Front will also move like rear

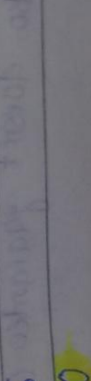
We achieve circularity by using MOD

Procedure

Similar as For Front

Non-deletable storage

He can't use space 2 as whenever r=f then queue should be empty (one space always remain empty)



Front

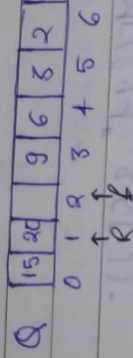
Rear

Rear	0	1	2	3	4	5	6
Front	0	1	2	3	4	5	6
size	7	7	7	7	7	7	7
value							

0	1	2	3	4	5	6
0	1	2	3	4	5	6

* Circular Queue guarantees that always the spaces will be reused.

size = 7



enqueue

void enqueue (struct Queue *q, int x)



* * Good For Full is that rear next is Full

if ((q->Rear+1) % q->size == q->Front)

printf("Queue is Full");

else

{ q->Rear = (q->Rear+1) % q->size;

q->Q[q->Rear] = x;

Dequeue

int Dequeue (struct Queue *q)

{ int x = -1;

if (q->Front == q->Rear)

printf("Queue is Empty");

else

{ q->Front = (q->Front+1) % q->size;

x = q->Q[q->Front];

}

→ void Display (struct Queue q)

{
 int i = q.Front + 1;

do

{

 printf("%d", q.Q[i]);

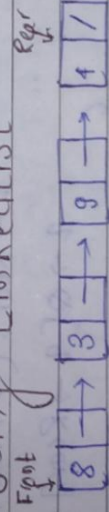
 i = (i+1) % q.size;

 while(i != (q.rear+1) % q.size);

 printf("%d", q.Q[i]);

}

Queue Using Linkedlist

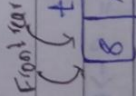


(i) Front Rear → When Queue is empty Front & Rear both NULL

Empty

if (Front == NULL)

special case



→ When only one node is present both Front & Rear will be on same node

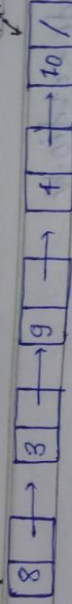
(ii) IF heap is full then No new Node will be created

Full

Node *t = new Node;

if (t == NULL)

Front
 ↓



enqueue()

```

void enqueue(int x)
{
    Node *t = new Node;
    IF (t = NULL)

```

PrintF("Queue is full");

```

    else
    {
        t->data = x;
        t->next = NULL;
    }

```

condⁿ For IF First Node is being inserted

```

IF (Front == NULL) Front = Rear = t;
else
{
    Rear->next = t;
    Rear = t;
}

```

Dequeue()

```

int dequeue()
{
    int x = -1; Node *p;
    IF (Front == NULL)
        PrintF("Queue is empty");
    else
    {
        p = Front;
        Front = Front->next;
        x = p->data;
        Free(p);
    }
    return x;
}

```

→ #include <stdio.h>
#include <stdlib.h>

struct Node

{
int data;
struct Node *next;
}* Front = NULL, *rear = NULL;

void enqueue (int x)

{
struct Node *t;

t = (struct Node *) malloc (sizeof (struct Node));

if (t == NULL)

printf ("Queue is Full\n");

else

{
t->data = x;

t->next = NULL;

if (Front == NULL)

Front = rear = t;

else

rear->next = t;

rear = t;

int dequeue ()

{
int x = -1;

struct Node *t;

```
IF (Front == NULL)
    printf("Queue is empty\n");
```

```
else
{
    r = Front -> data;
```

```
    t = Front;
```

```
    Front = Front -> next;
```

```
    Free(t);
```

```
    return x;
```

```
void Display()
```

```
{
    struct Node *p;
```

```
    while(p)
```

```
    {
        printf("%d", p->data);
```

```
        p = p->next;
```

```
    }
    int main()
```

```
{
    enqueue(10);
```

```
    enqueue(20);
```

```
    enqueue(30);
```

```
    enqueue(40);
```

```
    enqueue(50);
```

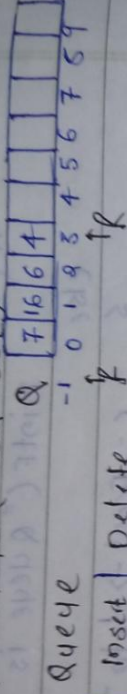
```
    Display();
```

```
    return 0;
```

```
}
```

→ 10, 20, 30, 40, 50

Double ended Queue (DEQUEUE)



Front	Insert	Delete
rear	X	✓
front	✓	X

DEQUEUE

	Insert	Delete
Front	✓	✓
rear	✓	✓

★ Both Front & rear can be used for insertion & deletion.

1/P Restricted

	Insert	Delete
Front	X	✓
rear	✓	✓

O/P Restricted

	Insert	Delete
Front	✓	✓
rear	✓	X

student exercise

1. Insert from Front & delete from Front
2. Insert from rear & delete from rear

Priority Queues

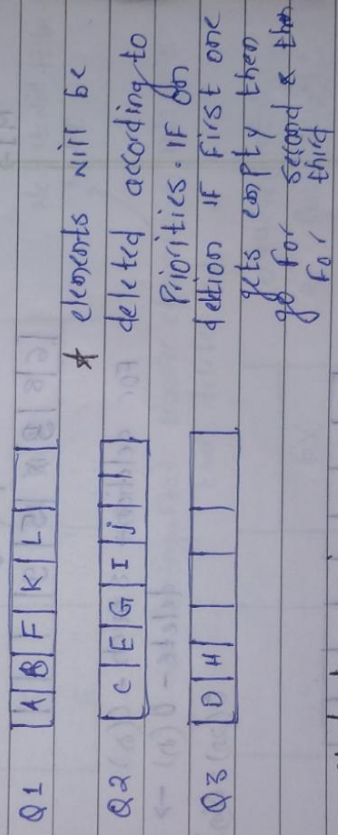
1. Limited set of priorities
2. Element priority

1. Limited Set of Priorities

Priorities = 3

element → A B C D E F G H I J K L
Priority → 1 1 2 3 2 1 2 3 2 2 1 1

Priority Queues



student exercise
↓
implement above

2. Element priority

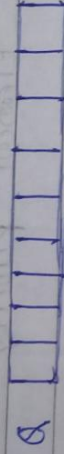
elements → 6, 8, 5, 10, 15, 2, 9, 17, 3, 8

↓
lowest

number highest priority → * you can change it abo

elements $\rightarrow 6, 8, 3, 10, 15, 3, 9, 17, 5, 8$

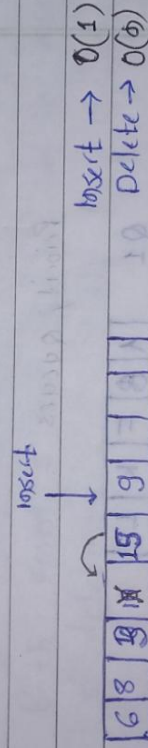
Lowest Number
highest priority



M1 $\rightarrow$ Insert in same order
Delete. Max priority by searching it

M2 $\rightarrow$ Insert in increasing order of priority
Delete last element of Array

M1 $\rightarrow$

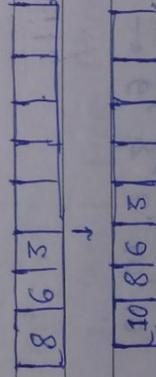


For deletion $\rightarrow$ search $= O(n)$
delete $= O(n) \rightarrow$ As we have to shift after deletion
 $O(20) = O(n)$

M2 $\rightarrow$



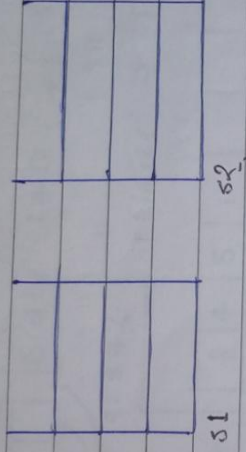
$\rightarrow$ Non shift 6 for inserting 8



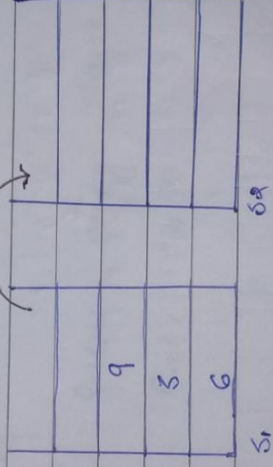
shift $= O(n)$ insert $\rightarrow O(n)$
delete $\rightarrow O(1)$

Queue Using 2 stacks

elements $\rightarrow 6, 3, 9, 5, 4, 2, 8, 12, 10, \dots$

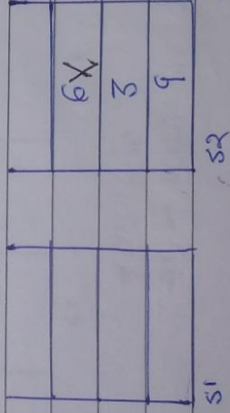


For inserting $\rightarrow$ Insert into s_1



Now it will follow FIFO

For deleting $\rightarrow$ First transfer element of s_1 into s_2 & then delete. Code



enqueue(int x)
 $\{$ push(s_1, x);
 $\}$

int dequeue()
 $\{$ int x = -1;
 $\}$

if (isEmpty(s_2))

$\{$ if (isEmpty(s_1))

$\{$ PF("Queue empty");

return x;

$\}$ else while (!isEmpty(s_1))

push($s_2, pop(s_1)$);

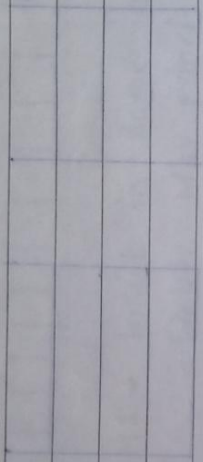
$\}$ return pop(s_2);
 $\}$

Student exercise

$\rightarrow$ Implement Queue using 2 stacks.

Ques 1) Find the value of x if $2x + 3 = 15$

$$2x + 3 = 15$$



12 9 6 3 0 -3 -6 -9 -12

For $2x + 3 = 15$ $\rightarrow$ $2x = 15 - 3$ $\rightarrow$ $2x = 12$ $\rightarrow$ $x = 6$



12 9 6 3 0 -3 -6 -9 -12

10. Find the value of x if $3x + 5 = 20$

For $3x + 5 = 20$ $\rightarrow$ $3x = 20 - 5$ $\rightarrow$ $3x = 15$ $\rightarrow$ $x = 5$

Check: $3(5) + 5 = 20$ $\rightarrow$ $15 + 5 = 20$ $\rightarrow$ $20 = 20$

3

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

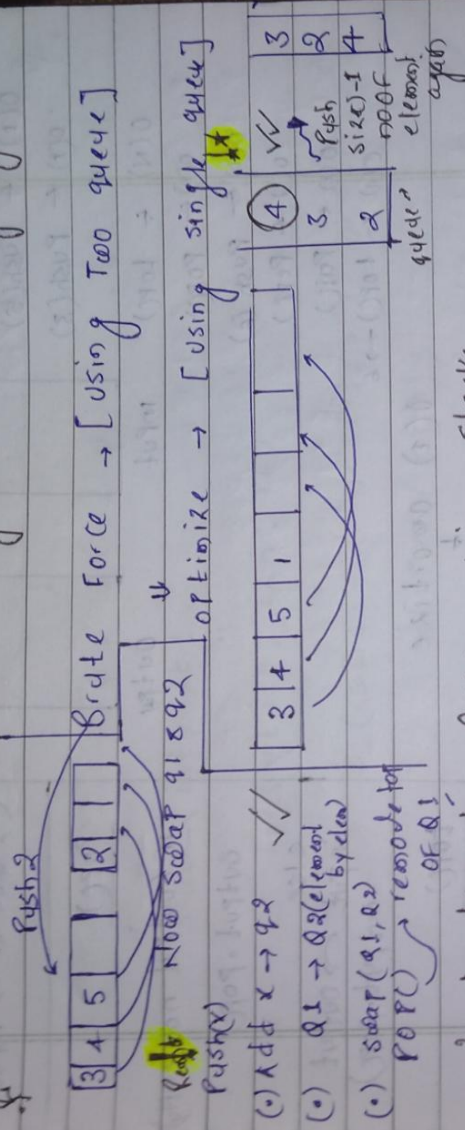
1

1

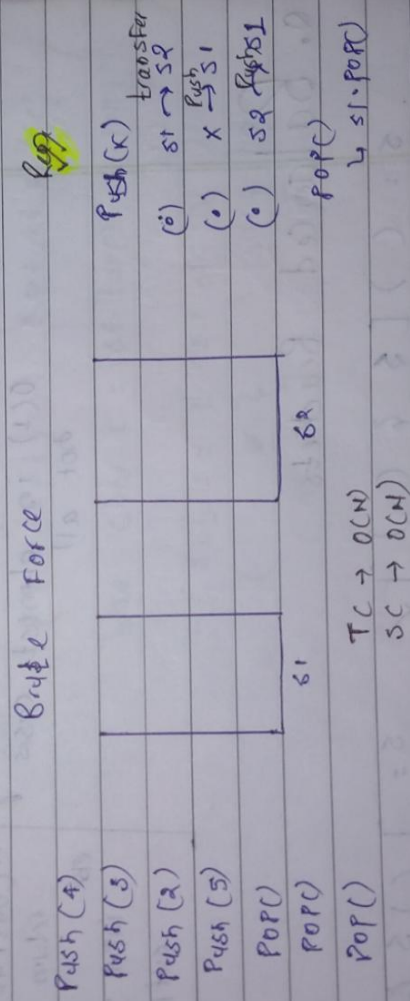
1

3. Deleted middle / element / of / a / stack

Q. 1. Implement stack using queue (using single queue)



2. Implement Queue using stacks



$O(1) \leftarrow \text{Push}(2)$
 $O(1) \leftarrow \text{Push}(5)$
 $O(2) \leftarrow \text{Push}(3)$

$O(N) \leftarrow \text{top}$

$O(1) \leftarrow \text{Pop}$

$O(1) \leftarrow \text{Push}(6)$

$O(1) \leftarrow \text{Pop}$

$O(2) \leftarrow \text{Pop}$

$O(N) \leftarrow \text{top} \rightarrow 6$

$O(1)$ amortize

means Push & Pop behaving

$O(1)$ is almost cases
not all

Q. Balanced Brackets

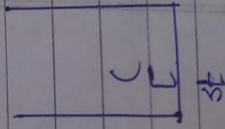
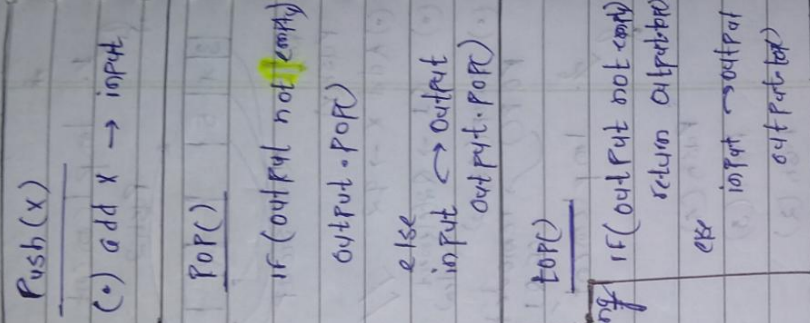
$S = () [\{ \} ()] \checkmark \quad S =] () \{ \} \times$

$S = [(]) \times$
 $S = [() [\{ () \}]]$

whenever get opening bracket push it into stack

close $\rightarrow]) \}$

if you get closing bracket check if stack is non-empty & match parenthesis



top

* s = [()]
↓ ↓ ↓ ↓ ↓
X
here stack should be empty
Keep care

s = [()]
↓
X

stack < int> st;

for (int i = 0; i < n; i++)

{
if (s[i] == '(' || s[i] == '[' || s[i] == '{')
st.push(s[i]);

else

if (st.empty()) return false;

else char c = st.top(), st.pop();

if (s[i] == ')' && c == '(' ||
(s[i] == '}' && c == '{') ||
(s[i] == ']' && c == '[')

else

return false;

if (st.empty())
return true;
return false;
For edge case
like "" "(" "

Q. Sort a stack

LRU Cache

Size = 2

(i) get (key) $\rightarrow$ value
 $\rightarrow -1$

Put (1, 1) $\rightarrow$ 1, 1, 1, 1

Put (2, 2) $\rightarrow$ 1, 1, 1, 1, 2, 2, 2, 2 (ii) Put (key, value)

get (1) $\rightarrow$ 1 $\rightarrow$ 1, 1, 1, 1, 2, 2, 2, 2

Put (3, 3) $\rightarrow$ 1, 1, 1, 1, 2, 2, 2, 2 Capacity Full

get (2) $\rightarrow$ -1 remove LRU

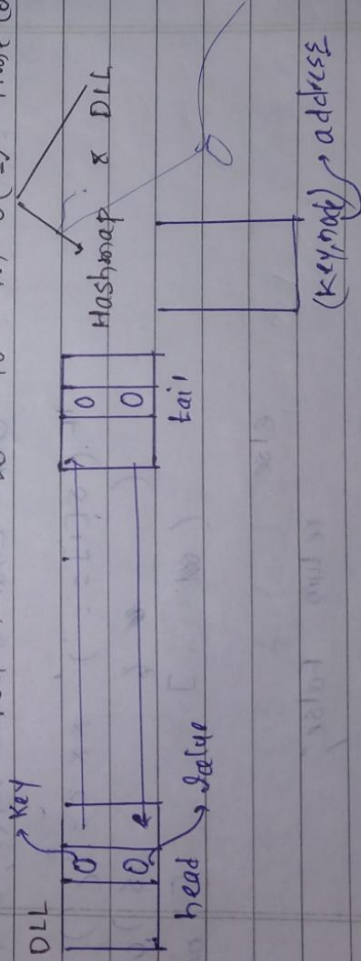
Put (4, 4) $\rightarrow$ 1, 1, 1, 1, 2, 2, 2, 2, 4, 4, 4, 4 then insert

get (1) $\rightarrow$ -1

get (3) $\rightarrow$ 3

get (4) $\rightarrow$ 4

You have to do in $O(1)$ Time Complexity



class LRUCache {
public:

class node {

public:

int key;

int val;

node* next;

node* prev;

node (int key, int val) {

key = key;

val = val;

}

node* head = new node(-1, -1);

node* tail = new node(-1, -1);

int cap;

unordered_map<int, node*> m;

LRUCache (int capacity) {

cap = capacity;

head → next = tail;

tail → prev = head;

}

void addnode (node* newnode) {

node* temp = head → next;

newnode → next = temp;

newnode → prev = head;

head → next = newnode;

temp → prev = newnode;

}

```
void deletenode (node* delnode) {
```

```
node* delprev = delnode -> prev;
```

```
node* delnext = delnode -> next;
```

```
delprev -> next = delnext;
```

```
delnext -> prev = delprev;
```

```
}
```

```
int get (int key) {
```

```
if (m.find(key) != m.end()) {
```

```
node* resnode = m[key];
```

```
int res = resnode -> val;
```

```
m.erase(key);
```

```
deletenode(resnode);
```

```
addnode(resnode);
```

```
m[key] = head -> next;
```

```
return res;
```

```
}
```

```
return -1;
```

```
}
```

```
void put (int key, int value) {
```

```
if (m.find(key) != m.end()) {
```

```
node* existnode = m[key];
```

```
m.erase(key);
```

```
deletenode(existnode);
```

```
}
```

Q.0 is a Gns station / circular tour (Retrol Pmap)

Date: / / Page:

ext;

if (size() == cap) {

erase (tail -> prev -> key);

deleteNode (tail -> prev);

addNode (new node (key, value));

key [key] = head -> next;

end();

key ->

tail;

next;

next;

end();

key ->

next;

tail;

head;

Q. 8

LRU

Size

$t(1,1)$

$t(2,1)$

$t(1,3)$

$t(1,4)$

$t(1,5)$

$t(1,6)$

$t(1,7)$

$t(1,8)$

$t(1,9)$

$t(1,10)$

$t(1,11)$

$t(1,12)$

$t(1,13)$

$t(1,14)$

$t(1,15)$

$t(1,16)$

$t(1,17)$

$t(1,18)$

$t(1,19)$

$t(1,20)$

Q. Implement Min, Max, stack

- 1) Nearest Greater to left
- 2) Nearest Greater to right
- 3) Nearest smaller to left
- 4) Nearest smaller to right

5. stock span Problem

6. Maximum area in histogram

7. Maximum area of rectangle in array

8. Rain water Tapping

9. Implementing with stack

10. Implementing stack using heap

11. The celebrity problem

12. Longest valid parenthesis

13. Iterative ToH

14. Next largest element or Nearest Greater right

15. Next largest element or Nearest Greater left

16. Next largest element or Nearest Greater

17. Next largest element or Nearest Greater

18. Next largest element or Nearest Greater

19. Next largest element or Nearest Greater

20. Next largest element or Nearest Greater

21. Next largest element or Nearest Greater

22. Next largest element or Nearest Greater

23. Next largest element or Nearest Greater

24. Next largest element or Nearest Greater

25. Next largest element or Nearest Greater

26. Next largest element or Nearest Greater

27. Next largest element or Nearest Greater

28. Next largest element or Nearest Greater

29. Next largest element or Nearest Greater

30. Next largest element or Nearest Greater

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

[1, 8, 2]

Next
greater
element

in ordinary

extra space
extra

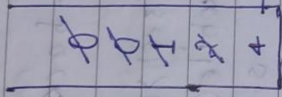
heap

Greater to

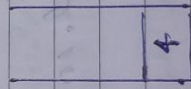
$O(N^2)$

⇕

③ 1 2 3 4



→ s.top() > arr[i],



vector s: [-1]

reverse(s.begin(), s.end())
return s

greater

store

push

vector <int> s;

stack <int> s;

for (int i = s.size() - 1; i >= 0; i--)

{

if (s.size() == 0) s.push_back(-1);

else if (s.size() > 0 && s.top() > arr[i])

{ s.push_back(s.top());

}

else if (s.size() > 0 && s.top() == arr[i])

{ while (s.size() > 0 && s.top() ==

arr[i]) s.pop();

if (s.size() == 0) s.push_back

else s.push_back(s.top());

}

s.push(arr[i]);

reverse(s.begin(), s.end());

return s;

Q. stock span problem

arr: [100 | 80 | 60 | 70 | 60 | 75 | 85]

Consecutive <= before it

[1 | 2 | 1 | 2 | 1 | 4 | 6]

back(-1)
> arr[i]

arr = arr[i]

if (s.top() < arr[i])

s.push(arr[i])

(s.top() < arr[i])

Count Consecutive smallest towards left

or equal to nearest smaller to left

Flag
check point

Q. Maximum rectangular area of Histogram

arr[] = {6, 3, 5, 4, 5, 1, 6, 8}

Output: 12

similar to above Q.
solved by using two function

① Consecutive - greater - right()

② Consecutive - greater - left()

every down

Code
vector<int> consecutive - smaller - left(C)
stack<pair<int, int>> s;
vector<int> res;

for (int i = 0; i < n; i++)

int cnt = 0;

if (s.size() == 0) {

res.push_back(i);

cnt = 1;

}

if greater

if store

if push

else if (s.top().first > price[i]) {

res.push_back(i);

cnt = i;

else {

while (s.size() > 0 &&

s.top().first < price[i])

cnt = s.top().second;

s.pop();

cnt++;

res.push_back(cnt);

s.push(price[i], cnt);

return res;

Q. Get minimum element from stack

Time $\rightarrow O(1)$

Space $\rightarrow O(1)$

int getMin()

{ if (s.size() == 0) { return -1; }

return minEle;

}

minEle
= 2

$$5(51)$$

Solid Push (inf x)

$$f'(s, \sin(\theta)) = 0$$

S. P4Sh (x)

$$\sin E|_c = X$$
$$1) \langle x \rangle = \langle x \rangle$$

2. $S. push(r)$

$$\text{else if } (x < \text{minEle})$$

50. Push (2*x - 10/Ek)

$$\min_{\lambda} \mathcal{L}(\lambda) = \lambda^*$$

Did you?

$$P(\text{size}) = \frac{1}{2} \text{Set} - \frac{1}{2}$$
$$r(\text{stop}) = \text{size}(e)$$

3. $S_{\text{POP}}(t)$

prog₂ $\text{errorf}(\text{S.toP}() < \text{minf}(\text{r}_k))$

min E = 2
S. 908C1

S. PopC

$$= 2 \times 10^4 \text{ kg} \cdot \text{s}^{-1}$$

int top()

{

if (s.size() == 0)

{ return -1; }

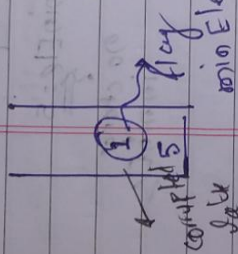
else

{ if (s.top() > minEle)

{ return s.top(); }

else if (s.top() < minEle)

{ return minEle; }

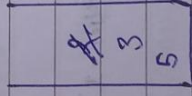


actual value

{

#

(1)



min Ele = 3 3 5

101 * after popping

2. we have to

go some how find min Ele i.e 3

Push



no change

Push

Pop

Pop

Q. ⁽¹⁰⁾ Revise Gas station / circular tour (Petrol Pump)

Page No. / /
Date: / /

3	5
---	---

MinEle = 83

at place of 3 push 2 x-minEle

1	5
---	---

convert data 1 stored [working as flag]

~~check point~~

Q. sliding window maximum

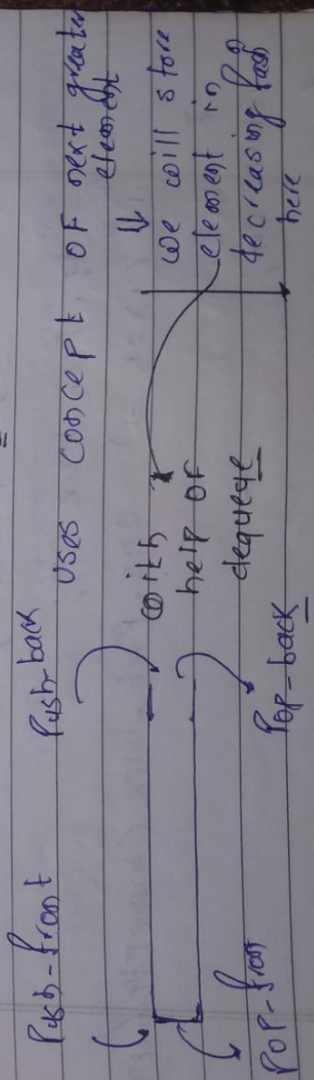
inputs = [1, 3, -1, -3, 5, 3, 6, 7], K=3

Output = [3, 3, 5, 5, 6, 7]

Naive solⁿ

$O(n \times K)$ $\rightarrow$ 1 2 3 4 5 6 7
1 3 -1 -3 5 3 6 7
Traverse every window

$\Rightarrow$ Optimize =



Gate 2014. radindra@gmail.com

1 3 -1 -5 5 3 6 7

storing index
1 3 4 5 6 7
at this point store index to

storing element in deque

1 2 3

0 1 2 3 4 5 6 7

7 6 5 4

int $\Rightarrow$ at any moment you reach any index $a[i] \geq a[j] < a[i+1]$ any element lesser than $a[i]$ can be no more maximum

vector<int> maxsliding (deque<int>, k)

deque<int> dq;
vector<int> ans;

for (int i=0; i<nums.size(); i++)

{ if (!dq.empty()) && dq.front() == i-k) dq.pop_front();

while (!dq.empty()) && nums[dq.back()] < nums[i] dq.pop_back();

dq.push_back(i);

if (i % k == 0) ans.push_back(dq.front());

return ans;

6

TC $\rightarrow O(N) + O(N)$
= $O(N)$
already reading out of bound $k \leq a[i]$

int cont
cons high

★ Four Pillars

- ① Plan
- ② knowledge
- ③ implement
- ④ Distribution

→ package

→ Consistency

→ Visionary idea

★ If you are not

planning you are planning to fail

★ Most of wars are won in general Tent

↓
Distribute to all, the knowledge

Placement procedure

less consistent
consider higher
pointer

MCQ → DS, Algo, Networking, DBMS, OS

Coding Round → Language + DS

Technical Round → 3 to 4 rounds

Interview + HR round

To know about you

Preparation strategy

very well

• Learn • DS • Competitive • Theory • Interviews

Language and Algo

Coding
↓
Technical Round

Learning Language (in MANIT)
Preferred (80-90%)

C/C++ Python Java

↓
any object oriented language [only one]

Most Important

Practice
it

~ 80% Questions

DS and Algo

(Techie De Light)

(In Terminus BIT/CODEFORCE)

* Geeks For Geeks

Array String

LinkedList

Dynamic Programming

Stack, Queue

Searching, Sorting

Tree, Graph, Hashing

Backtracking, greedy, Algo

loop

Array

String

Dynamic Program

IF you want to do this lekh

Package

dynamic programming is a dist

CODE TO COMPETE

→ Hacker rank → 30 days challenge for learning or revising

①

Patience

②

Determination

③

Enthusiasm

Necessary things for success

Daily 3 to 4 problems

↓

Consistency

*** Geeks For Geeks

Never ever leave

Learning Language Preferred (80-90%) in MANIT

C/C++ Python Java

↓
any Object oriented language - only one

Most Important

~ 80% Questions

DS and Algo

(Techie De Light)

(Interview BIT/CODEFORCE)

Array String,
LinkedList,

Dynamic programming

Stack, Queue

Searching, Sorting

Tree, Graph, Hashing

Backtracking, greedy, Algo

* Geeks For Geeks

① DS Algo

② Puzzle

③ Topic wise / Company

④ Interview

CODE TO COMPLETE

→ Hacker rank → 30 days challenge for learning or revising

① Patience

② Determination

③ Enthusiasm

Necessary things for success

*** Geeks For Geeks

Never ever leave

Daily 3 to 4 problems

↓
consistency

IF you want to this learn

Package dynamic programming is best

gifs to learn

→ 1. → 2. → 3.

4. 5.

Toc → Theory of computation

Page No. #
Date: / /

Theory

*(GATE VIDALAYA / JAVATPOINT)

6th semester

4th / 5th semester

Networking / O/S / OOPS

SQL / DBMS (Study Night.com) → best site

Software engineering / Digital electronics / TOC

5th / 4th semester

3rd semester

4th / 5th semester

Interview / Target

Puzzles

About yourself

Source

Interview BIT → Specially for fresh

GLASSDOOR

Interview Buddy

LeetCode

FOR JOBS

for open Camps

LINKDIN

1. Contact your seniors

2. Coding club

3. latest job update

For Freshers

4. Job Quest

5. All active jobs

NAUKART.COM

Company specific jobs

- ★ C.P → Competitive Programming
- ★ GFG → Geeks for Geeks

Don't do common mistake

1. Multiple language / large input
 ↪ Not go to multiple sites
 ↪ Not only learn
2. No Contemplation and Implementation
 ↪ what want from life
3. Chasing others - cop without concept
4. Giving up easily, No one to guide, loose consistency
 ↪ Not chase others by seeing

Do's

1. Repeat the Mistake
2. Revision
3. Make your own company / group → Not see toppers
 ↪ group of own level students
4. Never leave GFG

Sources / online

Theory things

1. Sanjit Jain
 Ragindra Rastogi

↪ for reading in short time

2. 5 Min Engg
 Easy Engg

languages

3. Saurabh Shukla
 Apni Kasha
 Jerry classes

4. GATE VIDYARATHI

JeeTA Point

GFG

Service OFFERED

1. Online DS classes
2. Certificates
3. Career Guidance

Quiz

Assignment

Talk

- * Even IF you don't do CP & Practice question ON GFG (Company wise) $\rightarrow$ it is more good

Competitive Coding

{ Tachic Delight < concept only >, spoj < practice only >, Interview Bit, Codeforces < advanced > }

1. Basics of programming language.
2. Time Complexity & space complexity
3. Searching
4. Vectors < imp for OT >

5. Sorting (insertion, selection, Bubble, Radix, Quick < imp >, Bucket, Heap < imp >)

6. Array and strings < imp for OT >
7. class & structure

7. Data structure

- a) Linked List < imp for INTERVIEW >
- b) Tree < imp >, Normal, B Tree, Red Black (Basics)

{ Tachic Delight, com }

- (C) Graphs (DFS, BFS, Dijkstra, Bellman, Prim's, Kruskal)
- { DFS & BFS code & { others concept only }

(d) stack & queues

Expt 8. Algorithms (To find prime no.s, permutation of strings)

9. Recursion

10. Dynamic programming

Object oriented programming

1. Inheritance

2. Encapsulation

3. Polymorphism

4. Etc..

DBMS: Study Tonight. Com &

Operating system: Study Tonight. Com & Silber-Schatz Book

Study Tonight. Com &

1. Types of Program (Imp)

2. Computer System Organisation

3. System Boot process (Imp)

4. Interrupts

5. OS structure [Multi programming,

Time sharing, Etc. - (Imp)]

6. System calls

7. Approach of OS Designed [Layered

Approach, Micro kernels, Modules]

8. OS Debugging (only simple concept)

9. process Management (Imp)

(a) What is process, process state

Process Control Block, scheduling queues,

schedulers, context switch

(b) process creation, termination

(c) IPC

(d) RPC

(e) Pipes

(f) Threads & Multithreading Models

9) Process synchronization (not much imp) Mutex & Semaphores (imp)

10. CPU scheduling Algo. (imp) and Thread scheduling

11. Deadlocks and their Detection & Handling Methods (very imp)

12. Memory Management (imp)

(a) swapping

(b) Paging

(c) Virtual Memory

13. Storage Management

(a) Storage Structure

(b) RAID (imp)

(c) Concept of Files

(d) File system & Examples

14. Protection & security [only overview]

15. Virtualization

Computer Networks:-

1. OSI Model & their detailed explanation

2. Protocols

a) IP

b) ICMP

c) ARP/RARP

d) TCP & UDP

e) HTTP, FTP, DHCP, DNS

3. Networking Devices

a) Hub

b) switch

c) Router

d) Multi layer switches

Optional
↑
SNMP, STP

(d) Fire walls [Optional]

(e) Load Balancers [Optional]

f. Routing protocols [only 02 exercises] [Optional]

a) RIP

b) OSPF

(c) EIGRP

5. Switching Topics <Very Imp For Network Based Organisation>

(a) How switch works

b) VLAN

c) Spanning Tree Protocol <Imp>

PQT -
HAR
Toc
UBMS
Algo
Comp
SOF
P. 55
for
interview

oops → 3rd sem

Ravindra babu ravula

Page No.

Date: / /

PQT → X → No Dhayan

After PQT TOC → In Fifth semes it is base of Compiler design but no need further

* DBMS → Not much expect from class → Youtube

Algorithm → बुके से पढ़ना → Abdul Bari

Computer Architecture → Interested in hardware → NP

Software engineering → Theory & Defn

OS → If time remaining

Imp. For

Interview Read DBMS From now